

Rezolvare Completă și Detaliată

Concursul Național pentru Ocuparea Posturilor Didactice

Anul 2017 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Titularizare Varianta 3 (12 iulie 2017)	2
SUBIECTUL I (30 de puncte)	2
1. Algoritmica submulțimilor	2
2. Proiectare baze de date: Normalizare	2
SUBIECTUL al II-lea (30 de puncte)	2
1. Programare: Eliminare nod din listă dublu înlanțuită	2
2. Algoritm eficient: Optimizare submulțime cu sume distincte	5

I. Rezolvare Titularizare Varianta 3 (12 iulie 2017)

[Subiect PDF](file:

SUBIECTUL I (30 de puncte)

1. Algoritmica submulțimilor

- **Backtracking:** Generarea tuturor submulțimilor unei mulțimi de N elemente prin vectori binari de lungime N (unde $x_i = 1$ dacă elementul i aparține submulțimii, și 0 altfel).
- **Complexitate:** $O(2^N)$ în timp.
- **Problemă (Generare submulțimi):**

```
#include <iostream>
using namespace std;
int x[20], n;
void afisare() {
    cout << "{ ";
    for (int i = 1; i <= n; ++i) if (x[i] == 1) cout << i << " ";
    cout << "}\n";
}
void backtrack(int k) {
    if (k > n) { afisare(); return; }
    for (int val = 0; val <= 1; ++val) {
        x[k] = val;
        backtrack(k + 1);
    }
}
```

—

2. Proiectare baze de date: Normalizare

- **Model conceptual:** Entități, instanțe, attribute, identificatori unici.
- **Forme normale:** 1FN (atomicitate), 2FN (fără dependențe parțiale), 3FN (fără dependențe tranzitive).

—

SUBIECTUL al II-lea (30 de puncte)

1. Programare: Eliminare nod din listă dublu înălțuită

Soluție C++:

```
#include <iostream>
#include <vector>
using namespace std;

struct Nod {
    int info;
    Nod *prec, *urm;
};

void elimin(Nod* &p, Nod* q) {
    if (q == nullptr) return;
    if (q->prec != nullptr) q->prec->urm = q->urm;
    if (q->urm != nullptr) q->urm->prec = q->prec;
    if (q == p) p = q->urm;
```

```

    delete q;
}

int main() {
    int val;
    vector<int> vals;
    while (cin >> val && val != 0) {
        vals.push_back(val);
    }
    if (vals.empty()) {
        cout << "nu exista\n";
        return 0;
    }

    int last_val = vals.back();
    Nod *p = nullptr, *u = nullptr;
    for (int x : vals) {
        Nod* nou = new Nod{x, u, nullptr};
        if (p == nullptr) { p = nou; u = nou; }
        else { u->urm = nou; u = nou; }
    }

    Nod* curr = p;
    while (curr != nullptr) {
        Nod* urm = curr->urm;
        if (curr->info == last_val) {
            elimin(p, curr);
        }
        curr = urm;
    }

    if (p == nullptr) {
        cout << "nu exista\n";
    } else {
        for (Nod* c = p; c != nullptr; c = c->urm) {
            cout << c->info << (c->urm == nullptr ? "" : " ");
        }
        cout << endl;
    }
    return 0;
}

```

Soluție Pascal:

```

program EliminareNoduri;
type
    PNod = ^TNod;
    TNod = record
        info: integer;
        prec, urm: PNod;
    end;
var
    p, u, nou, curr, urm_nod: PNod;
    val, last_val, i: integer;
    vals: array[1..100] of integer;
    count: integer;

```

```

procedure elimin(var p_list: PNod; q: PNod);
begin
    if q = nil then exit;
    if q^.prec <> nil then q^.prec^.urm := q^.urm;
    if q^.urm <> nil then q^.urm^.prec := q^.prec;
    if q = p_list then p_list := q^.urm;
    dispose(q);
end;

begin
    count := 0;
    read(val);
    while val <> 0 do
    begin
        count := count + 1;
        vals[count] := val;
        read(val);
    end;

    if count = 0 then
    begin
        writeln('nu exista');
        exit;
    end;

    last_val := vals[count];
    p := nil; u := nil;
    for i := 1 to count do
    begin
        new(nou);
        nou^.info := vals[i];
        nou^.prec := u;
        nou^.urm := nil;
        if p = nil then
        begin
            p := nou; u := nou;
        end
        else
        begin
            u^.urm := nou;
            u := nou;
        end;
    end;

    curr := p;
    while curr <> nil do
    begin
        urm_nod := curr^.urm;
        if curr^.info = last_val then
            elimin(p, curr);
        curr := urm_nod;
    end;

    if p = nil then
        writeln('nu exista')
    else

```

```

begin
    curr := p;
    while curr <> nil do
    begin
        write(curr^.info);
        if curr^.urm <> nil then write(' ');
        curr := curr^.urm;
    end;
    writeln;
end;
end.

```

2. Algoritm eficient: Optimizare submulțime cu sume distincte

Soluție C++:

```

#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    ifstream fin("titu.in");
    int n, k;
    if (!(fin >> n >> k)) return 0;
    vector<long long> even, odd;
    for (int i = 0; i < n; ++i) {
        long long val;
        fin >> val;
        if (val % 2 == 0) even.push_back(val);
        else odd.push_back(val);
    }
    fin.close();

    sort(even.rbegin(), even.rend());
    sort(odd.begin(), odd.end());

    int num_even = even.size();
    int num_odd = odd.size();
    vector<long long> pref_odd(num_odd + 1, 0);
    for (int i = 0; i < num_odd; ++i) pref_odd[i+1] = pref_odd[i] + odd[i];

    int max_total = 0;
    long long sum_even = 0;
    int limit = min(k, num_even);
    for (int e = 1; e <= limit; ++e) {
        sum_even += even[e-1];
        int st = 0, dr = num_odd, best_j = 0;
        while (st <= dr) {
            int mid = st + (dr - st) / 2;
            if (pref_odd[mid] < sum_even) {
                best_j = mid;
                st = mid + 1;
            } else {
                dr = mid - 1;
            }
        }
    }
}

```

```

        }
    }
    max_total = max(max_total, e + best_j);
}
cout << max_total << endl;
return 0;
}

```

Soluție Pascal:

```

program MaxElementeSelectate;
var
    fin: text;
    n, k, i, val, limit: integer;
    even, odd: array[1..10000] of int64;
    pref_odd: array[0..10000] of int64;
    num_even, num_odd: integer;
    sum_even: int64;
    max_total, e_val, st, dr, mid, best_j: integer;

procedure QuickSortEven(l, r: integer);
var i_idx, j_idx: integer; pivot, tmp: int64;
begin
    i_idx := l; j_idx := r; pivot := even[(l + r) div 2];
    repeat
        while even[i_idx] > pivot do i_idx := i_idx + 1;
        while even[j_idx] < pivot do j_idx := j_idx - 1;
        if i_idx <= j_idx then
            begin
                tmp := even[i_idx]; even[i_idx] := even[j_idx]; even[j_idx] := tmp;
                i_idx := i_idx + 1; j_idx := j_idx - 1;
            end;
    until i_idx > j_idx;
    if l < j_idx then QuickSortEven(l, j_idx);
    if i_idx < r then QuickSortEven(i_idx, r);
end;

procedure QuickSortOdd(l, r: integer);
var i_idx, j_idx: integer; pivot, tmp: int64;
begin
    i_idx := l; j_idx := r; pivot := odd[(l + r) div 2];
    repeat
        while odd[i_idx] < pivot do i_idx := i_idx + 1;
        while odd[j_idx] > pivot do j_idx := j_idx - 1;
        if i_idx <= j_idx then
            begin
                tmp := odd[i_idx]; odd[i_idx] := odd[j_idx]; odd[j_idx] := tmp;
                i_idx := i_idx + 1; j_idx := j_idx - 1;
            end;
    until i_idx > j_idx;
    if l < j_idx then QuickSortOdd(l, j_idx);
    if i_idx < r then QuickSortOdd(i_idx, r);
end;

begin
    assign(fin, 'titu.in'); reset(fin);
    read(fin, n, k);

```

```

num_even := 0; num_odd := 0;
for i := 1 to n do
begin
    read(fin, val);
    if val mod 2 = 0 then
    begin
        num_even := num_even + 1; even[num_even] := val;
    end
    else
    begin
        num_odd := num_odd + 1; odd[num_odd] := val;
    end;
end;
close(fin);

if num_even > 0 then QuickSortEven(1, num_even);
if num_odd > 0 then QuickSortOdd(1, num_odd);

pref_odd[0] := 0;
for i := 1 to num_odd do pref_odd[i] := pref_odd[i - 1] + odd[i];

max_total := 0; sum_even := 0;
if k < num_even then limit := k else limit := num_even;
for e_val := 1 to limit do
begin
    sum_even := sum_even + even[e_val];
    st := 0; dr := num_odd; best_j := 0;
    while st <= dr do
    begin
        mid := (st + dr) div 2;
        if pref_odd[mid] < sum_even then
        begin
            best_j := mid; st := mid + 1;
        end
        else dr := mid - 1;
    end;
    if e_val + best_j > max_total then max_total := e_val + best_j;
end;
writeln(max_total);
end.

```